

Einführung eines zentralen IT-Monitorings

Prototypische Implementierung mit Checkmk Raw Edition 2.3



PROJEKTDATEN & RAHMEN

Auftraggeber: Müller & Partner GmbH
Projektlaufzeit: 3 Projektstage (Proof of Concept)
Datum & Version: September / Oktober 2026 • Version 1.0
Zielerreichung: 11 von 11 MUSS-Kriterien erfüllt (100 %)
Systemauswahl: Checkmk Raw Edition 2.3 (Testsieger: 4,40 Pkt.)



PROJEKTTTEAM (GRONE UMSCHULUNGSTEAM)

Fachinformatiker Systemintegration

- Jan Kluth – Projektleitung, Ausgangslage & Fazit
- Marion Ballcke – Anforderungsanalyse & Marktanalyse
- Mathias Vorrau – Nutzwertanalyse & Entscheidungsmatrix
- Marco Schmidt – Netzwerkarchitektur & Custom Check
- Robert Ortmann – Störungssimulation & Datenanalyse

Agenda der Präsentation

Strukturierter 15-Minuten-Ablauf, aufgeteilt auf das Projektteam

01 Ausgangslage & Anforderungen

Manuelle Überwachung, Risiken & Pflichtenkatalog M01–M11 |  Jan Kluth & Marion Ballcke

02 Evaluation & Nutzwertanalyse

Vergleich von Checkmk, Zabbix und Nagios nach DIN-Matrix |  Mathias Vorrau

03 Technische Implementierung

Testnetzwerk, Dienst-Überwachung & eigener Python-Check |  Marco Schmidt

04 Validierung & Rollout-Fahrplan

4 Störungssimulationen, Datenanalyse & Empfehlung |  Robert Ortmann & Jan Kluth

Ausgangslage & Problemstellung

Warum die bisherige Betriebsweise ein untragbares Risiko für das Unternehmen darstellte

Das bisherige Problem: Manuelle Kontrolle

- X Wachsende IT: Mehrere Linux-Server für Web, Dateiverwaltung und DNS/DHCP.
- X Reaktives Brandlöschen: Admins erfuhren von Ausfällen erst durch Anrufe verärrgerter Mitarbeiter.
- X Unbemernte DNS/DHCP-Ausfälle: Ganze Abteilungen verloren unbemerkt die Netzwerkverbindung.
- X Drohender Datenverlust: Volllaufende Festplatten wurden vor dem Absturz nicht signalisiert.
- X Quälend langsame Antwortzeiten: Performance-Einbrüche blieben undokumentiert.

Das Projektziel: Proaktives Monitoring

- ✓ Früherkennung: Störungen erkennen und beheben, BEVOR Endanwender betroffen sind.
- ✓ Zentrales Web-Dashboard: Gesamter Zustand aller Hosts und Dienste auf einen Blick.
- ✓ Automatische Alarmierung: Sofortige Benachrichtigung bei Schwellwert-Überschreitung.
- ✓ 100 % Open Source: Etablierte Enterprise-Standards ohne teure Lizenzgebühren.
- ✓ Proof of Concept in 3 Tagen: Voll funktionsfähige Testumgebung zum Praxisnachweis.

Anforderungsanalyse (Abgabeprodukt A)

11 verbindliche MUSS-Kriterien zu 100 % erfüllt • Strukturierte Kategorien

1. Infrastruktur & Systeme

✓ M01: Zentrales Monitoring

Einrichtung einer zentralen Überwachungsinstanz

✓ M02: Multi-System-Setup

Mind. 2 Testsysteme (Linux & Windows)

✓ M03: Erreichbarkeit (ICMP)

Permanente Ping-Überwachung

✓ M04: Systemressourcen

CPU, RAM und Festplattenbelegung

2. Dienste & Schwellwerte

✓ M05: Netzwerkdienste

Aktive Überwachung: HTTP, MySQL, SSH

✓ M06: Definierte Grenzwerte

Fundierte Schwellen (WARN & CRIT)

✓ M09: Custom Check

Eigenes Python-Messwert-Skript (TCP)

✓ K04: Erweiterte Metriken

Zusätzliche historische Trendkurven

3. Alarmierung & Qualität

✓ M07: Automatische Alarmer

Sofortige Warnung bei Grenzwertüberschreitung

✓ M08: Zentrales Dashboard

Taktische Web-Übersicht für Administratoren

✓ M10: Störungssimulation

4 reale Ausfälle gezielt ausgelöst & validiert

✓ M11: Abnahmetests

Strukturierte Testmatrix (13/13 bestanden)

 **ZUSÄTZLICH 3 KANN-ANFORDERUNGEN UMGESETZT (60 %):**

K01 E-Mail-Alarmierung vorbereitet • K02 Erweiterte Dashboards & Matrizen • K04 Detaillierter Custom Check mit Performancedaten

Die 3 Kandidaten im Systemvergleich

Marktanalyse führender Open-Source-Lösungen für den 3-tägigen Proof of Concept

Checkmk Raw Edition 2.3

TESTSIEGER 🏆

- Open Source (GNU GPLv2)
- Sehr intuitive Web-GUI (WATO)
- Automatische Service-Erkennung (Discovery)
- Fertige Dashboards out-of-the-box
- Keine externe SQL-Datenbank nötig
- Schnellster Time-to-Value für 3 Tage

Zabbix 7.0 LTS

RANG 2 (MÄCHTIG)

- Open Source (GPLv2)
- Sehr flexibel & extrem skalierbar
- Mächtige Template-Engine
- ⚠ Externe SQL-DB zwingend nötig
- ⚠ Hohe Konfigurationskomplexität
- ⚠ Setup-Aufwand zu hoch für 3 Tage

Nagios Core 4.5

RANG 3 (KLASSIKER)

- Open Source (GPLv2)
- Bewährter, historischer Standard
- Extrem sparsam bei Ressourcen
- ✗ Veraltete Weboberfläche
- ✗ Rein textbasierte Konfig-Dateien
- ✗ Keine integrierten Dashboards

Nutzwertanalyse & Entscheidungsmatrix

Fundierte Entscheidung nach DIN-Bewertungsmethode (Skala 1 bis 5)

		Kriterium		Gewicht	Checkmk Raw	Zabbix	Nagios Core
1. Kosten / Lizenz (Open Source)	10 %	5 / 0,50	5 / 0,50	5 / 0,50			
2. Bedienbarkeit & GUI (Ausschlaggebend)	20 %	5 / 1,00 🏆	3 / 0,60	2 / 0,40			
3. OS-Unterstützung (Linux & Windows)	10 %	4 / 0,40	5 / 0,50	4 / 0,40			
4. Netzwerkdienste (HTTP, SQL, SSH)	15 %	4 / 0,60	5 / 0,75	4 / 0,60			
5. Alarmierung & Benachrichtigung	15 %	4 / 0,60	4 / 0,60	3 / 0,45			
6. Visualisierung & Dashboards (Ausschlaggebend)	15 %	5 / 0,75 🏆	4 / 0,60	2 / 0,30			
7. Erweiterbarkeit & Dokumentation	15 %	4 / 0,55	5 / 0,65	4 / 0,60			
GESAMTERGEBNIS (PUNKTE / RANG)	100 %	4,40 (RANG 1) 🏆	4,20 (RANG 2)	3,30 (RANG 3)			

Warum Checkmk gewonnen hat: Der Gamechanger

Detailanalyse des Punkteabstands zwischen Checkmk (4,40) und Zabbix (4,20)



Wo Checkmk siegte: +0,55 Punkte Vorsprung

Bedienbarkeit & GUI (20 % Gewicht):

- Checkmk holte die Bestnote 5 / 5 (1,00 Nutzwert)
 - Zabbix erhielt nur 3 / 5 (0,60 Nutzwert)
- Allein hier: +0,40 Punkte Vorsprung für Checkmk!

Visualisierung & Dashboards (15 % Gewicht):

- Checkmk: 5 / 5 (0,75 Nutzwert) durch fertige Dashboards
 - Zabbix: 4 / 5 (0,60 Nutzwert)
- Weitere +0,15 Punkte Vorsprung für Checkmk!

Fazit: In den beiden praxisrelevantesten Kriterien dominierte Checkmk klar.



Zabbix' Aufholjagd: +0,35 Punkte – warum es nicht reichte

Zabbix war in 3 Kriterien überlegen:

- Netzwerkdienste (15 %): Zabbix 5 vs. Checkmk 4 → +0,15
- Erweiterbarkeit (15 %): Zabbix 5 vs. Checkmk 4 → +0,10
- OS-Support (10 %): Zabbix 5 vs. Checkmk 4 → +0,10

Warum es trotzdem nicht zum Gesamtsieg reichte:

- Zabbix holte insgesamt +0,35 Punkte Vorsprung auf.
- Checkmk sicherte sich bei GUI & Dashboards aber +0,55 Punkte!

Differenz-Rechnung:

+0,55 (Checkmk) minus +0,35 (Zabbix) = +0,20 Punkte Gesamtsieg!

Testnetz-Architektur & Topologie

Strukturiertes 3-System-Testnetz (VirtualBox Host-Only Subnetz 192.168.56.0/24)

mon-srv01 (MONITORING-SERVER)

IP: 192.168.56.101

- OS: Ubuntu 26.04 LTS VM
- Rolle: Checkmk Raw Edition 2.3
- Betrieb: Docker-Container (restart: always)
- Web-GUI: Port 80 (gemappt auf Host: 8080)
- Status: 24 Services aktiv überwacht

web-srv01 (LINUX WEBSERVER)

IP: 192.168.56.102

- OS: Geklonte Ubuntu VM (4 GB RAM)
- Dienste: Apache2 (HTTP 80), MariaDB (SQL 3306), SSH (22)
- Checkmk Agent: Port 6556 TCP aktiv
- Custom Check: Python TCP-Verbindungsüberwachung
- Ports: SSH 2223 / HTTP 8081

win-client01 (WINDOWS CLIENT)

IP: 192.168.56.1

- OS: Windows 10/11 Host-PC
- Agent: Checkmk Windows Agent (MSI)
- Dienst: Checkmk Service aktiv
- Firewall: Port 6556 freigegeben
- Überwacht: CPU, RAM, Festplatte C:

KOMMUNIKATIONSPRINZIP (PULL-MONITORING):

Checkmk Server pollt minütlich Port 6556 aller Zielsysteme → Agent liefert Sensordaten → Livestatus Core bewertet Schwellwerte → Live-Dashboard aktualisiert.

Umgesetzte Dienst-Überwachung & Schwellwerte

Aktive Überwachung von Anwendungsdiensten (M05) & definierte Grenzwerte (M06)

Dienst / Ressource		Überwacher Service	Port	Warnung (WARN)	Kritisch (CRIT)	Live-Status
Apache Webserver	HTTP_Apache	Port 80	> 2,0 Sekunden	> 5,0 s / Ausfall	OK (200 OK)	●
MariaDB Datenbank	MySQL_Database	Port 3306	> 1,0 Sekunde	> 3,0 s / Ausfall	OK (Port 3306)	●
Fernwartung	SSH_Service	Port 22	Verzögerung > 1s	Ausfall / Refused	OK (Port 22)	●
Custom Check	Active_TCP_Connections	-	> 100 Verbindungen	> 200 Verbindungen	OK (1 Verb.)	●
CPU-Auslastung	CPU utilization	-	> 80 % (5 Min)	> 95 % (5 Min)	OK (3–7 %)	●
Arbeitsspeicher	Memory	-	> 85 % genutzt	> 95 % genutzt	OK (19 %)	●
Festplatte	Filesystem /	-	> 85 % belegt	> 95 % belegt	OK (37 %)	●

Eigenentwicklung: Custom Local Check (M09)

Entwicklung eines eigenständigen Python-Plugins zur TCP-Verbindungsanalyse

```
#!/usr/bin/env python3
# Local Check: active_tcp_connections.py
import subprocess, sys

def get_tcp_connections():
    out = subprocess.check_output(['ss', '-ant'], text=True)
    count = sum(1 for line in out.splitlines() if 'ESTAB' in line)

    warn, crit = 100, 200
    status = 0
    status_txt = 'OK'

    if count >= crit:
        status, status_txt = 2, 'CRIT'
    elif count >= warn:
        status, status_txt = 1, 'WARN'

    print(f'{status} "Active_TCP_Connections" '
          f'connections={count};{warn};{crit}'
          f'{status_txt} - {count} aktive TCP Verbindungen')

if __name__ == '__main__':
    get_tcp_connections()
```



Funktionsweise & Checkmk-Protokoll

Speicherort: /usr/lib/check_mk_agent/local/

Protokoll-Format: <STATUS> "<NAME>" <METRIKEN> <TEXT>

Status-Codes:

- 0 = OK (Normalbetrieb unter 100 Verbindungen)
- 1 = WARN (Erhöhte Last ab 100 Verbindungen)
- 2 = CRIT (Kritische Last / DoS-Verdacht ab 200)

Besonderer Mehrwert:

- Durch das Mitliefern der Metrik 'connections=X;100;200' generiert Checkmk automatisch RRD-Performance-Graphen im Dashboard.
- Beweist die grenzenlose Erweiterbarkeit von Checkmk.

Störungssimulation & Validierung (M10)

4 reale Ausfallszenarien simuliert und Reaktionszeiten protokolliert

#	Szenario	Auslösebefehl	Erwartung	Tatsächliche Erkennung	Reaktionszeit
1	Apache Web-Ausfall	<code>sudo systemctl stop apache2</code>	HTTP → CRIT	CRIT (Connection refused)	47 Sek. ⚡
2	CPU-Volllast	<code>stress --cpu 4 --timeout 180</code>	CPU → WARN/CRIT	WARN (62s) → CRIT (75s)	62 Sek. ⚡
3	Festplatte voll	<code>dd if=/dev/zero of=/tmp/file</code>	Disk → CRIT	CRIT (Belegung > 95%)	60 Sek. ⚡
4	Host-Totalausfall	VM Power Off (VirtualBox)	Host → DOWN	DOWN (ICMP Ping Fail)	127 Sek. ⚡

FAZIT DER STÖRUNGSTESTS:

- 100 % Erkennungsrate aller Ausfälle in durchschnittlich unter 65 Sekunden.
- Automatische Entwarnung: Nach Behebung sprangen alle Checks ohne manuelles Zutun wieder auf OK (Grün) zurück.

Datenanalyse & Schwellwert-Validierung (Abgabeprodukt I)

Untersuchung der Messkurven und Bestätigung der praxistauglichen Grenzwerte

Messgröße 1: CPU-Auslastung

- Normalbetrieb: 5 bis 15 % Grundlast.
- Störfall: Steiler Anstieg auf 98–100 %.
- Validierung: Schwelle bei 80 % mit 5-Minuten-Zeitfenster filtert kurzzeitige Update-Spitzen zuverlässig heraus.
- Ergebnis: Kein Fehlalarm-Risiko, maximaler Schutz.

Messgröße 2: Festplattenbelegung

- Normalbetrieb: Flacher Verlauf bei ca. 37 %.
- Störfall: Steiler Füllkurvenanstieg auf über 95 %.
- Validierung: Warnstufe bei 85 % lässt Admins mehrere Tage Reaktionszeit für Log-Bereinigungen.
- Ergebnis: Verhindert Datenverlust und Dienststillstand.

Messgröße 3: HTTP-Verfügbarkeit

- Normalbetrieb: Stabile Antwortzeiten unter 15 ms.
- Störfall: Sofortiger Ausschlag auf Connection Refused.
- Validierung: Schwelle bei 2,0s (Warn) und 5,0s (Crit) schützt Kunden vor quälend langsamen Ladezeiten.
- Ergebnis: Essentiell für geschäftskritische Webdienste.

Herausforderungen & Lessons Learned

Praxisprobleme während des Projekts und wie wir sie gelöst haben

❌ Problem: Paketkonflikte auf Ubuntu 26.04 → ✅ Lösung: Offizieller Docker-Container

- Neuere Perl-Versionen blockierten das native .deb-Paket von Checkmk.
- Lösung: Wir setzten Checkmk als Docker-Container auf – isoliert, stabil und mit automatischem Neustart bei Systemboot.

❌ Problem: MariaDB Netzwerk-Bindung → ✅ Lösung: Bind-Address auf 0.0.0.0

- MariaDB lauschte standardmäßig nur auf 127.0.0.1 (Localhost).
- Lösung: Konfiguration in 50-server.cnf auf 0.0.0.0 angepasst, sodass der Port 3306 über das Host-Only-Netzwerk sauber überwacht werden kann.

❌ Problem: Windows-Agent Firewall-Freigabe → ✅ Lösung: Port 6556 TCP Rule

- Der Windows-Agent lieferte zunächst keine Daten an den Linux-Server.
- Lösung: Port 6556 TCP im Host-Only-Netzwerk in der Windows Defender Firewall explizit freigegeben.

Projektergebnis & Zielerreichung (Abgabeprodukt J)

Alle 11 MUSS-Anforderungen zu 100 % erfüllt • Strukturierter Abnahmetest bestanden

MUSS-ANFORDERUNGEN

11 / 11

100 % erfüllt

KANN-ANFORDERUNGEN

3 / 5

60 % umgesetzt

STÖRUNGSPRÜFUNGEN

4 / 4

100 % erkannt

ABNAHMETESTS (T01–T12)

13 / 13

100 % bestanden



Fazit des Proof of Concept:

- ✓ Die Checkmk Raw Edition 2.3 hat bewiesen, dass sie die Anforderungen der Müller & Partner GmbH vollständig erfüllt.
- ✓ Der Zeitrahmen von 3 Projekttagen (geplant: 39,0 h | Ist: 41,5 h) wurde mit einer minimalen Abweichung von +6,4 % eingehalten.
- ✓ Die Lösung arbeitet ressourcenschonend, benötigt keine teuren Lizenzen und ist sofort einsatzbereit.

Empfehlung & 3-Phasen-Rollout-Fahrplan

Konkrete Handlungsempfehlung für die Geschäftsführung zur Produktivübernahme

PHASE 1 (WOCHE 1–2)

Pilotbetrieb auf Kernsystemen

- Installation auf dedizierter Hardware/VM.
- Anbindung der 10 kritischsten Server (Active Directory, DNS, ERP, File-Server).
- Einarbeitung des internen IT-Teams in die WATO-Administration.

PHASE 2 (WOCHE 3–4)

Vollständiger Rollout

- Erfassung aller Arbeitsplatz-PCs und NAS-Speicher über Windows/Linux-Agenten.
- Einrichtung der E-Mail- und SMS-Benachrichtigungsketten.
- Feintuning der Schwellwerte für den Normalbetrieb.

PHASE 3 (MONAT 2)

Erweiterung & Netzinfrastruktur

- Anbindung von Netzwerkschwitches und Routern via SNMP.
- Zentrales Log-Monitoring und Auswertung historischer Daten.
- Kapazitätsplanung für zukünftiges Hardware-Wachstum.



Vielen Dank für Ihre Aufmerksamkeit!

Wir stehen nun für Ihre Fragen und die Live-Demonstration zur Verfügung.



LIVE-DEMONSTRATION BEREIT:

- Checkmk Web-GUI: <http://localhost:8080/cmk/> | Login: cmkadmin / cmkadmin
- Live-Systeme: mon-srv01 (.101) • web-srv01 (.102) • win-client01 (.1)
- Live-Störungstest auf Abruf bereit: systemctl stop apache2 (HTTP CRIT)